智能小车系统架构与开发指南

版本：1.1

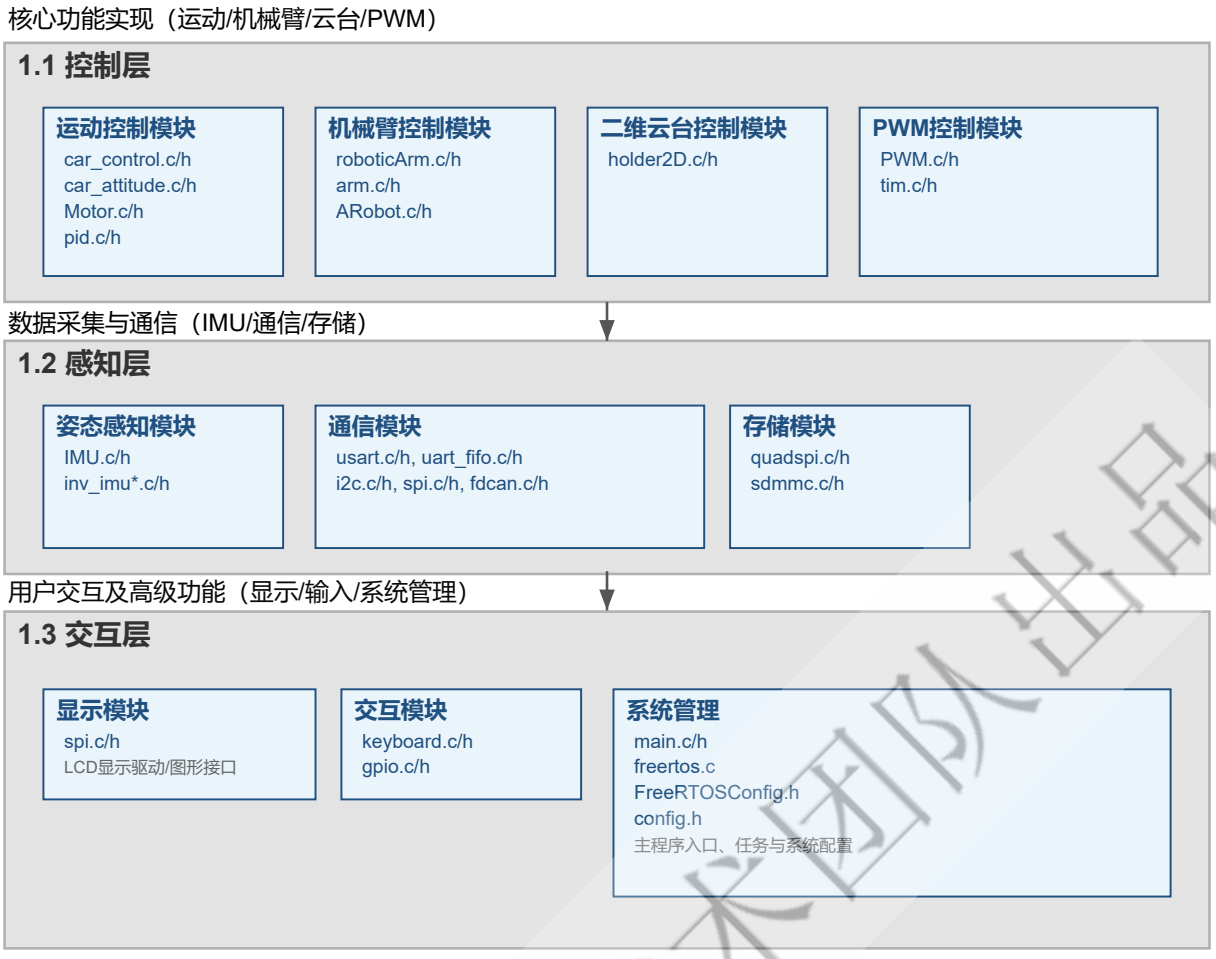
目录

- 智能小车系统架构与开发指南
  - 目录
  - 1. 代码结构概述
    - 1.1 控制层
    - 1.2 感知层
    - 1.3 交互层
  - 2. FreeRTOS控制层详解
    - 2.1 任务架构
    - 2.2 调度机制
    - 2.3 任务同步与通信
    - 2.4 关键任务执行
    - 2.5 资源管理
    - 2.6 开发与调试
  - 3. 通信协议与接口
    - 3.1 串口通信协议
    - 3.2 蓝牙通信协议
    - 3.3 SCServo舵机通信协议
  - 4. API使用示例
    - 4.1 基本功能示例
    - 4.2 进阶功能示例
  - 5. 开发与调试指南
    - 5.1 开发环境配置
    - 5.2 固件烧录
    - 5.3 调试技巧
  - 附录A：关键数据结构
  - 附录B：修订历史

Innolegend技术团队出品

1. 代码结构概述

智能小车底层代码采用模块化设计，分为控制层、感知层和应用层，便于开发和维护。



1.1 控制层

控制层是智能小车的核心功能实现层，主要负责运动控制、机械臂控制、二维云台控制等执行功能。

- 运动控制模块
  - `car_control.c/h`：实现小车的定点、直线行走和旋转功能
  - `car_attitude.c/h`：管理小车姿态控制
  - `Motor.c/h`：电机驱动控制
  - `pid.c/h`：PID算法实现，支持位置式和增量式PID
- 机械臂控制模块
  - `roboticArm.c/h`：6自由度机械臂控制
  - `arm.c/h`：机械臂基础功能
  - `ARobot.c/h`：机械臂高级功能
- 二维云台控制模块
  - `holder2D.c/h`：二维云台控制，支持精确角度控制和位置设定
- **PWM控制模块**

Innolegend技术团队出品

- `PWM.c/h`：PWM信号生成，用于电机调速和舵机控制
- `tim.c/h`：定时器配置，为PWM和中断提供基础

1.2 感知层

感知层负责收集和处理各类传感器数据，为控制层提供决策依据。

- **姿态感知模块**
  - `IMU.c/h`：IMU数据处理和姿态解算
  - `inv_imu*.c/h`：IMU驱动和数据处理库
- **通信模块**
  - `usart.c/h`：串口通信
  - `uart_fifo.c/h`：串口FIFO缓冲区管理
  - `i2c.c/h`：I2C通信协议
  - `spi.c/h`：SPI通信协议
  - `fdcan.c/h`：CAN总线通信
- **存储模块**
  - `quadspi.c/h`：QuadSPI闪存接口
  - `sdmmc.c/h`：SD卡接口

1.3 交互层

应用层提供用户交互界面和高级功能实现。

- **显示模块**
  - `spi.c/h`：LCD显示驱动和图形接口
  - `oled_spi_V0.2.c`：OLED显示驱动和图形接口
  - 支持文字、数字、图形和图像显示
- 交互模块
  - `keyboard.c/h`：按键输入处理
  - `gpio.c/h`：GPIO管理，控制LED和蜂鸣器
- **系统管理**
  - `main.c/h`：主程序入口和系统初始化
  - `freertos.c`：FreeRTOS任务管理
  - `FreeRTOSConfig.h`：FreeRTOS配置
  - `config.h`：系统全局配置

Innolegend技术团队出品

2. FreeRTOS控制层详解

智能小车采用FreeRTOS实时操作系统作为核心控制层，实现多任务并行处理、精确时序控制和资源管理，确保各功能模块高效协同工作。

2.1 任务架构

FreeRTOS控制层将小车功能划分为多个独立任务，每个任务负责特定功能模块，通过优先级调度和同步机制实现协同工作：

任务名称	优先级	周期(ms)	功能描述
LEDBlink	osPriorityAboveNormal	1000	系统心跳指示，LED闪烁
_9axisService	osPriorityNormal3	5	IMU数据采集与姿态解算
CarAttitude	osPriorityNormal5	10	小车姿态与运动控制
UartHandle	osPriorityNormal7	1	串口通信处理
Key	osPriorityNormal6	1	按键输入处理
Buzzer	osPriorityNormal1	1	蜂鸣器声音控制
Screen_LCD	osPriorityNormal1	100	LCD显示更新
AppTaskCreate	osPriorityNormal1	1	应用任务创建

用户可以根据自身需求创建更多的任务，以下是两种创建任务的方式： **1.通过Cubemx软件自动创建任务：**

```
/* Definitions for LEDBlink */
osThreadId_t LEDBlinkHandle;
uint32_t LEDBlinkBuffer[ 128 ];
osStaticThreadDef_t LEDBlinkControlBlock;
const osThreadAttr_t LEDBlink_attributes = {
  .name = "LEDBlink",
  .cb_mem = &LEDBlinkControlBlock,
  .cb_size = sizeof(LEDBlinkControlBlock),
  .stack_mem = &LEDBlinkBuffer[0],
  .stack_size = sizeof(LEDBlinkBuffer),
  .priority = (osPriority_t) osPriorityAboveNormal,
};
void Task_LEDBlink(void *argument);
LEDBlinkHandle = osThreadNew(Task_LEDBlink, NULL,
&LEDBlink_attributes);//osThreadNew属于CMSIS-RTOS v2标准通用接口
```

2.手动创建任务：

Innolegend技术团队出品

```
void Task_APP();
xTaskCreate(Task_APP,"Test",512,NULL,(osPriority_t)
osPriorityNormal5,NULL);//FreeRTOS原生API
```

另外本套代码提供通过外部硬件按钮动态创建应用任务。 如`Task_APP_test()`，用于实现特定功能测试和演示。系统可使用`key_callback1`回调函数,按下用户所想要使用的按键可创建名为`Task_APP_test`任务：

```
// 在回调函数中创建任务
void key_callback1()
{
    //按下按键后在Task_Key任务中可以调用该函数进行任务的创建
    xTaskCreate(Task_APP_test, "APP", configMINIMAL_STACK_SIZE * 2, NULL,
osPriorityNormal2, NULL);
}

// 测试任务功能—删除任务自身
void Task_APP_test()
{
    vTaskDelete(NULL);
}
```

2.2 调度机制

FreeRTOS采用抢占式调度机制，确保高优先级任务能及时响应：

- **时间片轮转**：同优先级任务通过时间片轮转方式共享CPU时间
- **优先级抢占**：高优先级任务可随时抢占低优先级任务执行
- **精确延时**：支持`osDelay`和`vTaskDelayUntil`实现精确时序控制

系统配置参数：

- 系统时钟：480MHz
- 时钟节拍：1000Hz (1ms)
- 最大优先级：56级
- 堆内存大小：81920字节

2.3 任务同步与通信

FreeRTOS提供多种机制实现任务间同步与通信：

1. 信号量

信号量使用示例：

Innolegend技术团队出品

```
// 等待信号量
if (osSemaphoreAcquire(semphr_uart_receiveHandle, portMAX_DELAY) == osOK) {
    // 处理串口数据
}

// 释放信号量
osSemaphoreRelease(semphr_buzzer_triggerHandle);
```

2. 消息队列

系统使用`queue_key`消息队列传递按键事件：

```
// 发送按键消息
char key = '1';
osMessageQueuePut(queue_keyHandle, &key, 0, 0);

// 接收按键消息
char key;
if (xQueueReceive(queue_keyHandle, &key, portMAX_DELAY)) {
    // 处理按键事件
}
```

3. 任务通知

任务间可通过任务通知进行轻量级通信，无需额外的同步对象：

```
// 发送任务通知
xTaskNotify(targetTaskHandle, 0x01, eSetBits);

// 等待任务通知
uint32_t notificationValue;
xTaskNotifyWait(0, 0xFFFFFFFF, &notificationValue, portMAX_DELAY);
```

2.4 关键任务执行

1. IMU数据采集与姿态解算任务

`Task_9axisService`以5ms周期执行，确保姿态数据实时更新以保证小车正常行驶需要的姿态数据：

```
void Task_9axisService(void *argument)
{
  TickType_t xLastWakeTime;
  const TickType_t xFrequency = TASK_ITV_IMU; // 5ms
  xLastWakeTime = xTaskGetTickCount();
```

Innolegend技术团队出品

```
  for(;;)
  {
    vTaskDelayUntil(&xLastWakeTime, xFrequency);
    IMU_getYawPitchRoll(ypr);
    IMU_TT_getgyro(motion6);
    Car_Attitude_Yaw_Update(car_attitude.current_v_angle, TASK_ITV_IMU*0.001);
  }
}
```

2. 小车姿态与运动控制任务

`Task_CarAttitude`以10ms周期执行，更新输入并更新输出以实现闭环控制：

```
void Task_CarAttitude(void *argument)
{
  TickType_t xLastWakeTime;
  const TickType_t xFrequency = TASK_ITV_CAR; // 10ms
  xLastWakeTime = xTaskGetTickCount();

  for(;;)
  {
    vTaskDelayUntil(&xLastWakeTime, xFrequency);
    Tim_Update_Enc_Speed();      // 更新编码器速度
    Is_Speed_Enc_Update = 1;     // 设置LCD显示标志位
    Motor_Update_Input_All();    // 更新电机输入
    Car_Attitude_Update_Input();// 更新姿态输入
    Car_Control_Update_Input(); // 更新控制输入
    Car_Control_Update_Output();// 更新控制输出
    Car_Attitude_Update_Output();// 更新姿态输出
    Motor_Update_Output_All();  // 更新电机输出
  }
}
```

3. OLED显示任务

`Task_Screen`以100ms周期更新显示内容，展示系统状态：

```
void Task_Screen(void *argument)
{

  for(;;)
  {
    OLED_ShowString(0, 0, "Hello World!");
    osDelay(100);
  }

}
```

Innolegend团队出品

2.5 资源管理

FreeRTOS控制层对系统资源进行统一管理，确保多任务环境下的安全访问：

1. 内存管理

- 静态内存分配：关键任务使用静态分配方式，确保内存安全
- 动态内存分配：应用任务使用动态分配，灵活创建临时任务

2. 中断管理

- 中断优先级：系统配置最低中断优先级为15，最高为5
- 中断安全API：高优先级中断中只能调用中断安全的FreeRTOS API

3. 任务栈管理

每个任务分配128个32位字（512字节）栈空间，确保任务执行不发生栈溢出：

```
uint32_t LEDBlinkBuffer[128];
osStaticThreadDef_t LEDBlinkControlBlock;
```

2.6 开发与调试

FreeRTOS控制层提供丰富的调试支持：

1. 断言机制

系统配置断言处理，在关键错误发生时停止系统：

```
#define configASSERT(x) if ((x) == 0) {taskDISABLE_INTERRUPTS(); for( ;; );}
```

2. 任务状态监控

支持获取任务运行状态、栈使用情况等信息：

```
// 获取任务状态
eTaskState state;
state = eTaskGetState(taskHandle);

// 获取栈高水位线
UBaseType_t highWaterMark;
highWaterMark = uxTaskGetStackHighWaterMark(NULL);
```

3. 性能优化

Innolegend技术团队出品

通过合理设置任务优先级和执行周期，优化系统性能：

- 姿态解算（5ms周期）：确保控制精度
- 运动控制（10ms周期）：平衡响应速度和系统负载
- 显示更新（100ms周期）：降低非关键任务频率，减轻系统负担

3. 通信协议与接口

3.1 串口通信协议

小车与上层处理系统之间采用自定义串口通信协议：

- **波特率**：115200bps
- **数据位**：8位
- **停止位**：1位
- **校验位**：无

数据帧格式：

```
X123Y456Z789
```

- 命令字：X,Y,Z，定义数据类型和操作
- 数据：根据命令字定义的数据内容，X为X坐标，Y为Y坐标，Z为偏航角

3.2 蓝牙通信协议

蓝牙通信采用esp32-wroom的BLE蓝牙功能，通过esp32-wroom通过串口与主控芯片连接，波特率可自定义。

```
void Task_Broadcast(void *argument)
{
  /* USER CODE BEGIN Task_Broadcast */
  /* Infinite loop */
  for(;;)
  {
        char buf[64];

        char* msg = "HelloFromStation\n";
        HAL_UART_Transmit(&huart4, (uint8_t*)msg, strlen(msg), 100);//Send To BT

    osDelay(50);
  }
  /* USER CODE END Task_Broadcast */
}
```

Innolegend技术团队出品

Task_Broadcast 是一个 FreeRTOS 任务，用于 周期性通过蓝牙（UART4）发送数据: 默认每 50ms 发送一次 "HelloFromStation\n"。 可修改`Task_Broadcast`任务，发送 传感器数据、控制指令、调试信息 等。

3.3 SCServo舵机通信协议

机械臂和二维云台使用SCServo串行总线舵机，波特率：1000000bps，具体内容参考飞特官网：飞特串行舵通讯协议

4. API使用示例

4.1 基本功能示例

1. 系统初始化

```
/* MPU Configuration--------------------------------------------------------*/
MPU_Config();

/* MCU Configuration--------------------------------------------------------*/

/* Reset of all peripherals, Initializes the Flash interface and the Systick. */
HAL_Init();

/* USER CODE BEGIN Init */
init_motor();//电机初始化
/* USER CODE END Init */

/* Configure the system clock */
SystemClock_Config();

/* USER CODE BEGIN SysInit */

/* USER CODE END SysInit */

/* Initialize all configured peripherals */
MX_GPIO_Init();
MX_TIM2_Init();
MX_TIM3_Init();
MX_TIM4_Init();
MX_TIM5_Init();
MX_TIM6_Init();
MX_USB_OTG_FS_PCD_Init();
MX_QUADSPI_Init();
MX_SPI4_Init();
MX_TIM13_Init();
MX_UART4_Init();
MX_USART1_UART_Init();
MX_SPI2_Init();
MX_USART2_UART_Init();
MX_USART3_UART_Init();
MX_UART5_Init();
```

Imolegend技术团队出品

```
  /* USER CODE BEGIN 2 */
  OLED_Init();
  MusicPlayer_Init();
  HAL_Delay(1000);
  OLED_DrawBMP(9,0,119,8,Genshin);  //LOGO;
  HAL_Delay(1000);
  OLED_Clear();
  delay_init(CPU_FREQ);     //重新初始化延时函数, Parameter: CPU_FREQ
  IMU_init();
  HAL_Delay(1000);
  HAL_TIM_Encoder_Start(&htim2, TIM_CHANNEL_ALL); //开启TIM2编码器
  HAL_TIM_Encoder_Start(&htim3, TIM_CHANNEL_ALL); //开启TIM3编码器
  HAL_TIM_Encoder_Start(&htim4, TIM_CHANNEL_ALL); //开启TIM4编码器
  HAL_TIM_Encoder_Start(&htim5, TIM_CHANNEL_ALL); //开启TIM5编码器

  HAL_TIM_Base_Start_IT(&htim2);                  //开启TIM2更新中断
  HAL_TIM_Base_Start_IT(&htim3);                  //开启TIM3更新中断
  HAL_TIM_Base_Start_IT(&htim4);                  //开启TIM4更新中断
  HAL_TIM_Base_Start_IT(&htim5);                  //开启TIM5更新中断
  HAL_TIM_Base_Start_IT(&htim6);

  //小车控制初始化
  init_Car_Attitude();
  init_Car_Contorl();
  HAL_Delay(2000);
  printf("init ok");
  setup_roboticArm();//机械臂初始化
  HAL_Delay(1000);
  initArm();//机械臂位置初始化
  HAL_Delay(1000);

  /* USER CODE END 2 */

  /* Init scheduler */
  osKernelInitialize();

  /* Call init function for freertos objects (in cmsis_os2.c) */
  MX_FREERTOS_Init();

  /* Start scheduler */
  osKernelStart();

  /* We should never get here as control is now taken by the scheduler */
```

2. LED任务示例

```
/* USER CODE END Header_Task_LEDBlink */
void Task_LEDBlink(void *argument)
{
  /* USER CODE BEGIN Task_LEDBlink */
  /* Infinite loop */
```

Innolegend技术团队出品

```
  for(;;)
  {
    HAL_GPIO_TogglePin(LED0_GPIO_Port, LED0_Pin);
    osDelay(1000);
  }
  /* USER CODE END Task_LEDBlink */
}
```

具体代码可见附录代码包`freertos.c`。

3. 串口接收任务示例

```
/* USER CODE BEGIN Header_Task_Uart4Handle */
/**
* @brief Function implementing the Uart4Handle thread.
* @param argument: Not used
* @retval None
*/
/* USER CODE END Header_Task_Uart4Handle */
void Task_Uart4Handle(void *argument)
{
  /* USER CODE BEGIN Task_Uart4Handle */
  char frame[UART_FRAME_MAX_LEN];
    /* Infinite loop */
    for(;;)
    {
        memset(frame, 0, sizeof(frame));
        // 从队列拿到一帧数据（由中断回调送入队列）
        if (xQueueReceive(uart4_frame_queue, frame, portMAX_DELAY) == pdPASS) {
        // Todo:解析帧数据

        }
    }
  /* USER CODE END Task_Uart4Handle */
}
```

具体代码可见附录代码包`freertos.c`。

4. 按键任务示例

```
/* USER CODE BEGIN Header_Task_Key */
/**
* @brief Function implementing the Key thread.
* @param argument: Not used
* @retval None
*/
/* USER CODE END Header_Task_Key */
void Task_Key(void *argument)
{
```

Innolegend技术团队出品

```
    char key;
    for(;;)
    {
      if (xQueueReceive(queue_keyHandle, &key, portMAX_DELAY)) {
        OLED_ShowChar(0, 0, key);
          //Todo: 处理按键事件
    }
        osDelay(1);
    }
}
```

具体代码可见附录代码包freertos.c、keyboard.c,该任务须配合vKeyScanTask按键扫描任务,按键经过软件消抖处理,并通过队列在任务间传递所按下按键的数据信息。

5. 音乐播放器任务示例

```
/* USER CODE BEGIN Header_Task_Buzzer */
/**
* @brief Function implementing the Buzzer thread.
* @param argument: Not used
* @retval None
*/
/* USER CODE END Header_Task_Buzzer */
void Task_Buzzer(void *argument)
{
  /* USER CODE BEGIN Task_Buzzer */
  /* Infinite loop */
  for(;;)
  {
    PlayNote(M1, 100); // 播放音符 M1, 持续时间 100ms
    osDelay(1);
  }
  /* USER CODE END Task_Buzzer */
}
```

具体代码可见附录代码包freertos.c、MusicPlayer.c

6. Screen显示任务示例

```
/* USER CODE BEGIN Header_Task_Screen */
/**
* @brief Function implementing the Screen thread.
* @param argument: Not used
* @retval None
*/
/* USER CODE END Header_Task_Screen */
void Task_Screen(void *argument)
{
  /* USER CODE BEGIN Task_Screen */
```

Innolegend技术团队出品

```
  /* Infinite loop */
  for(;;)
  {
    OLED_ShowString(0, 0, "Hello World!");
    osDelay(100);
  }
  /* USER CODE END Task_Screen */
}
```

具体代码可见附录代码包`freertos.c`、`oled_spi_V0.2.c`、`spi.c`文件。其中`oled_spi_V0.2.c`文件包含了OLED屏显示的一系列接口函数,`spi.c`文件包含了LCD屏显示的一系列接口函数。API接口函数示例如下： **6.1. OLED显示API函数**

```
/**
 * @brief OLED显示屏的硬件初始化函数
 */
void OLED_Init(void);

/**
 * @brief 用于清空OLED屏幕显示内容的函数（所有像素点关闭）
 */
void OLED_Clear(void);//更新显示,可清屏刷新内容,可用可不用

/**
 * @brief 用于在OLED屏幕的指定位置显示一个ASCII字符，支持两种字体大小：
          8x16 像素（SIZE == 16）
          6x8 像素（默认小字体）
 * @param x 字符的起始列坐标（0~127）
 * @param y 字符的起始页坐标（0~7）
 * @param chr 要显示的ASCII字符
 */
void OLED_ShowChar(u8 x, u8 y, char chr);

/**
 * @brief 用于在OLED屏幕的指定位置显示一个无符号整数
 * @param x 数字的起始列坐标（0~127）
 * @param y 数字的起始页坐标（0~7）
 * @param num 要显示的数字（范围：0~4294967295）
 * @param len 数字的显示位数（如 123 的 len 可以是 3 或更大）
 * @param size2 字体大小（8=6x8 小字体，16=8x16 大字体）
 */
void OLED_ShowNum(u8 x, u8 y, u32 num, u8 len, u8 size2);

/**
 * @brief 用于在OLED屏幕的指定位置显示一个有符号整数（支持负数），主要特性包括：
自动处理负号（-）
右对齐显示（左侧补空格）
可变数字长度
两种字体大小（8x16 或 6x8 像素）
 * @param x 数字的起始列坐标（0~127）
 * @param y 数字的起始页坐标（0~7）
```

Intelegent技术团队出品

```
 * @param num 要显示的数字（范围：0~4294967295）
 * @param len 数字的显示位数（如 123 的 len 可以是 3 或更大）
 * @param size2 字体大小（8=6x8 小字体，16=8x16 大字体）
 */
void OLED_ShowSignedNum(u8 x, u8 y, int32_t num, u8 len, u8 size2);

/**
 * @brief 用于在OLED屏幕的指定位置显示一个汉字，核心特性包括：
依赖预定义汉字字库（Hzk 数组）
固定16x16像素大小（2页x16列）
支持自定义字库索引
基于SPI逐字节写入
 * @param x 汉字左上角的列坐标（0~127）
 * @param y 汉字左上角的页坐标（0~7，实际占用2页）
 * @param no 汉字在字库 Hzk 中的索引号（需确保字库已正确初始化）
 */
void OLED_ShowChinese(u8 x, u8 y, u8 no);

/**
 * @brief 用于控制OLED显示屏的颜色显示模式，核心功能是切换正常显示和反色显示两种模式：
正常模式（i=0）：黑色背景 + 白色像素（默认）
反色模式（i=1）：白色背景 + 黑色像素
 * @param i 模式选择：
           0 = 正常显示（黑底白字）
           1 = 反色显示（白底黑字）
 */
void OLED_ColorTurn(u8 i);

/**
 * @brief 用于控制OLED屏幕的显示方向，可实现180度旋转显示，核心功能包括：
正常方向（i=0）：默认显示方向
倒置方向（i=1）：屏幕内容旋转180度
 * @param i 方向选择：
           0 = 正常方向（默认）
           1 = 倒置方向（旋转180°）
 */
void OLED_DisplayTurn(u8 i);



//更多API请参考oled_spi_V0.2.c文件
```

6.2. LCD显示API函数

```
/**
 * @brief LCD初始化
 */
void SPI_LCD_Init(void);

/**
 * @brief 清屏
```

Innolegend技术团队出品

```
 */
void LCD_Clear(void);

/**
 * @brief 设置显示方向
 * @param direction 方向枚举 (Direction_H, Direction_V, Direction_HV,
Direction_VH)
 */
void LCD_SetDirection(uint8_t direction);

/**
 * @brief 设置前景色
 * @param Color 颜色值 (如 LCD_RED, LCD_GREEN, LCD_BLUE)
 */
void LCD_SetColor(uint32_t Color);

/**
 * @brief 设置背景色
 * @param Color 颜色值
 */
void LCD_SetBackColor(uint32_t Color);

/**
 * @brief 显示ASCII字符串
 * @param x 起始x坐标
 * @param y 起始y坐标
 * @param p 字符串指针
 */
void LCD_DisplayString(uint16_t x, uint16_t y, char *p);

/**
 * @brief 显示数字
 * @param x 起始x坐标
 * @param y 起始y坐标
 * @param number 要显示的整数
 * @param len 显示位数
 */
void LCD_DisplayNumber(uint16_t x, uint16_t y, int32_t number, uint8_t len);

/**
 * @brief 显示小数
 * @param x 起始x坐标
 * @param y 起始y坐标
 * @param number 要显示的小数
 * @param len 总显示位数
 * @param decs 小数位数
 */
void LCD_DisplayDecimals(uint16_t x, uint16_t y, double number, uint8_t len,
uint8_t decs);

/**
 * @brief 绘制矩形
 * @param x 左上角x坐标
 * @param y 左上角y坐标
```

Innoregend技术团队出品

```
 * @param width 宽度
 * @param height 高度
 */
void LCD_DrawRect(uint16_t x, uint16_t y, uint16_t width, uint16_t height);

/**
 * @brief 绘制填充圆
 * @param x 圆心x坐标
 * @param y 圆心y坐标
 * @param r 半径
 */
void LCD_FillCircle(uint16_t x, uint16_t y, uint16_t r);

// 更多API请参考 spi.h 文件
```

4.2 进阶功能示例

1. QSPI Flash使用示例

```
/*****************************************************************************
******************
*   函 数 名: QSPI_W25Qxx_Test
*   入口参数: 无
*   返 回 值: QSPI_W25Qxx_OK - 测试成功并通过
*   函数功能: 进行简单的读写测试, 并计算速度
*   说    明: 无
*****************************************************************************
*****************/

#define W25Qxx_NumByteToTest    32*1024                 // 测试数据的长度, 64K

int32_t QSPI_Status_Test ;      //检测标志位

uint32_t W25Qxx_TestAddr  = 0   ;                           // 测试地址
uint8_t  W25Qxx_WriteBuffer_Test[W25Qxx_NumByteToTest];     //  写数据数组
uint8_t  W25Qxx_ReadBuffer_Test[W25Qxx_NumByteToTest];      //  读数据数组

int8_t QSPI_W25Qxx_Test(void)       //Flash读写测试
{
    uint32_t i = 0; // 计数变量
    uint32_t ExecutionTime_Begin;       // 开始时间
    uint32_t ExecutionTime_End;     // 结束时间
    uint32_t ExecutionTime;             // 执行时间
    float    ExecutionSpeed;            // 执行速度

// 擦除 >>>>>>>

    ExecutionTime_Begin     = HAL_GetTick();    // 获取 systick 当前时间, 单位ms
    QSPI_Status_Test            = QSPI_W25Qxx_BlockErase_32K(W25Qxx_TestAddr);  //
擦除32K字节
    ExecutionTime_End       = HAL_GetTick();    // 获取 systick 当前时间, 单位ms
```

Innolegend技术团队出品

```
    ExecutionTime = ExecutionTime_End - ExecutionTime_Begin; // 计算擦除时间，单位
ms

    if( QSPI_Status_Test == QSPI_W25Qxx_OK )
    {
        printf ("\r\nW25Q256 擦除成功，擦除32K字节所需时间：%d
ms\r\n",ExecutionTime);
    }
    else
    {
        printf ("\r\n 擦除失败!!!!!  错误代码:%d\r\n",QSPI_Status_Test);
        while (1);
    }

// 写入 >>>>>>>

    for(i=0;i<W25Qxx_NumByteToTest;i++)  //先将数据写入数组
    {
        W25Qxx_WriteBuffer_Test[i] = i;
    }
    ExecutionTime_Begin     = HAL_GetTick();    // 获取 systick 当前时间，单位ms
    QSPI_Status_Test                =
QSPI_W25Qxx_WriteBuffer(W25Qxx_WriteBuffer_Test,W25Qxx_TestAddr,W25Qxx_NumByteToTe
st); // 写入数据
    ExecutionTime_End       = HAL_GetTick();    // 获取 systick 当前时间，单位ms

    ExecutionTime  = ExecutionTime_End - ExecutionTime_Begin;       // 计算擦除时
间，单位ms
    ExecutionSpeed = (float)W25Qxx_NumByteToTest / ExecutionTime ; // 计算写入速
度，单位 KB/S
    if( QSPI_Status_Test == QSPI_W25Qxx_OK )
    {
        printf ("\r\n写入成功,数据大小：%d KB，耗时：%d ms，写入速度：%.2f
KB/s\r\n",W25Qxx_NumByteToTest/1024,ExecutionTime,ExecutionSpeed);
    }
    else
    {
        printf ("\r\n写入错误!!!!!  错误代码:%d\r\n",QSPI_Status_Test);
        while (1);
    }

// 读取 >>>>>>>

    printf
("\r\n*************************************************************************
************************\r\n");
    ExecutionTime_Begin     = HAL_GetTick();    // 获取 systick 当前时间，单位ms
    QSPI_Status_Test                =
QSPI_W25Qxx_ReadBuffer(W25Qxx_ReadBuffer_Test,W25Qxx_TestAddr,W25Qxx_NumByteToTest
);  // 读取数据
    ExecutionTime_End       = HAL_GetTick();    // 获取 systick 当前时间，单位ms

    ExecutionTime  = ExecutionTime_End - ExecutionTime_Begin;                //
```

Innolegend技术团队出品

```
计算擦除时间，单位ms
    ExecutionSpeed = (float)W25Qxx_NumByteToTest / ExecutionTime / 1024 ;   // 计
算读取速度，单位 MB/S

    if( QSPI_Status_Test == QSPI_W25Qxx_OK )
    {
        printf ("\r\n读取成功,数据大小: %d KB, 耗时: %d ms, 读取速度: %.2f MB/s
\r\n",W25Qxx_NumByteToTest/1024,ExecutionTime,ExecutionSpeed);
    }
    else
    {
        printf ("\r\n读取错误!!!!!  错误代码:%d\r\n",QSPI_Status_Test);
        while (1);
    }

// 数据校验 >>>>>>>

    for(i=0;i<W25Qxx_NumByteToTest;i++) //验证读出的数据是否等于写入的数据
    {
        if( W25Qxx_WriteBuffer_Test[i] != W25Qxx_ReadBuffer_Test[i] )   //如果数据
不相等，则返回0
        {
            printf ("\r\n数据校验失败!!!!!\r\n");
            while(1);
        }
    }
    printf ("\r\n校验通过!!!!! QSPI驱动W25Q64测试正常\r\n");


// 读取整片Flash的数据，用以测试速度 >>>>>>>
    printf
("\r\n*****************************************************************************
*************************\r\n");
    printf ("\r\n上面的测试中，读取的数据比较小，耗时很短，加之测量的最小单位为ms，计算
出的读取速度误差较大\r\n");
    printf ("\r\n接下来读取8M的数据用以测试速度，这样得出的速度误差比较小\r\n");
    printf ("\r\n开始读取>>>>\r\n");
    ExecutionTime_Begin     = HAL_GetTick();    // 获取 systick 当前时间，单位ms

    for(i=0;i<W25Qxx_FlashSize/(W25Qxx_NumByteToTest);i++)  // 每次读取
W25Qxx_NumByteToTest 字节的数据
    {
    QSPI_Status_Test         =
QSPI_W25Qxx_ReadBuffer(W25Qxx_ReadBuffer_Test,W25Qxx_TestAddr,W25Qxx_NumByteToTest
);
        W25Qxx_TestAddr = W25Qxx_TestAddr + W25Qxx_NumByteToTest;
    }
    ExecutionTime_End       = HAL_GetTick();    // 获取 systick 当前时间，单位ms

    ExecutionTime  = ExecutionTime_End - ExecutionTime_Begin;                  //
计算擦除时间，单位ms
    ExecutionSpeed = (float)W25Qxx_FlashSize / ExecutionTime / 1024 ;   // 计算读
取速度，单位 MB/S
```

Innogend技术团队出品

```
    if( QSPI_Status_Test == QSPI_W25Qxx_OK )
    {
        printf ("\r\n读取成功,数据大小: %d MB, 耗时: %d ms, 读取速度: %.2f MB/s
\r\n",W25Qxx_FlashSize/1024/1024,ExecutionTime,ExecutionSpeed);
    }
    else
    {
        printf ("\r\n读取错误!!!!!  错误代码:%d\r\n",QSPI_Status_Test);
        while (1);
    }

    return QSPI_W25Qxx_OK ;  // 测试通过
}
```

具体代码以及更多API接口函数可见附录代码包`quadspi.c`

2. 小车运动轨迹规划 (方形轨迹)

```
void Task_SquarePath(void *argument)
{
    Set_Car_Control(100, 0, 0);    // 前进100mm
    osDelay(2000);                 // 等待完成
    Set_Car_Control(0, 0, 90);     // 左转90度
    osDelay(1500);                 // 等待完成
    Set_Car_Control(100, 0, 0);    // 前进100mm
    osDelay(2000);                 // 等待完成
    Set_Car_Control(0, 0, 90);     // 左转90度
    osDelay(1500);                 // 等待完成
    Set_Car_Control(100, 0, 0);    // 前进100mm
    osDelay(2000);                 // 等待完成
    Set_Car_Control(0, 0, 90);     // 左转90度
    osDelay(1500);                 // 等待完成
    Set_Car_Control(100, 0, 0);    // 前进100mm
    osDelay(2000);                 // 等待完成
    Set_Car_Control(0, 0, 90);     // 左转90度
    osDelay(1500);                 // 等待完成

    vTaskDelete(NULL);
}
```

具体代码以及更多API接口函数可见附录代码包`freertos.c`、`car_control.c`、`car_attitude.c`、`motor.c`或者资料包中的文件智能小车-运动控制API详解

3. 机械臂预设动作（伪代码）

```
void Task_ArmActions(void *argument)
{
    arm_init();
```

Intelegent技术团队出品

```
    osDelay(500);

    // 抓取动作
    arm_grab_action();
    osDelay(3000);

    // 放置动作
    arm_place_action();
    osDelay(3000);

    // 挥手动作
    arm_wave_action();
    osDelay(3000);

    vTaskDelete(NULL);
}
```

具体代码以及更多API接口函数可见附录代码包`freertos.c`、`roboticArm.c`、或者资料包中的文件智能小车-机械臂控制API详解

4. 云台跟踪 (伪代码)

```
void Task_GimbalTracking(void *argument)
{
    setup_holder();
    osDelay(500);

    // 假设从上层系统获取目标坐标 (target_x, target_y)
    while(1)
    {
        // 获取目标坐标
        get_target_coordinates(&target_x, &target_y);

        // 控制云台指向目标
        set_xy_position(target_x, target_y);

        osDelay(50); // 控制频率
    }
}
```

具体代码以及更多API接口函数可见附录代码包`freertos.c`、`holder2D.c`、或者资料包中的文件智能小车-二维云台API详解

5. 开发与调试指南

5.1 开发环境配置

- **IDE**: STM32CubeIDE 或Keil

Innolegend技术团队出品

- **配置工具**: STM32CubeMX
- **调试器**: ST-Link V2
- **编译器**: ARM GCC

5.2 固件烧录

1. 连接ST-Link调试器到电脑和底板的SWD接口
2. 在Keil中配置调试器
3. 编译项目
4. 点击"Debug"或"Run"按钮进行烧录

5.3 调试技巧

- **串口打印**: 使用`printf`函数通过串口 **(默认重定义fputc使用uart1)** 输出调试信息
- **IDE调试**: 利用STM32CubeIDE的调试功能，设置断点、观察变量、单步执行
- **FreeRTOS调试**: 使用FreeRTOS提供的调试工具或插件，查看任务状态、栈使用情况、CPU占用率等
- **LED指示**: 利用LED指示灯显示程序运行状态或错误代码
- **逻辑分析仪**: 使用逻辑分析仪观察GPIO、I2C、SPI等信号波形

附录A：关键数据结构

```
// PID控制器结构体
typedef struct {
    float p;
    float i;
    float d;
    float set;
    float real;
    float err;
    float err_last;
    float integral;
    float out;
    float out_limit;
} pid;

// 小车控制结构体
typedef struct {
    float target_x;
    float target_y;
    float target_angle;
    float current_x;
    float current_y;
    float current_angle;
    pid pid_line_pos;
    pid pid_line_speed;
    pid pid_spin;
```

Innolegend技术团队出品

```
    car_mode mode;
} car_control_t;
```

附录B：修订历史

版本	日期	说明
1.0	2025-05-30	初始版本1.0
1.1	2025-08-20	改进版本1.1

Innolegend技术团队出品